

RV-MalTrace: Hardware-Assisted Behavior Tracing for RISC-V Linux Workloads

Anonymous Authors

Abstract—Malware behavior analysis at scale requires transparent, low-perturbation observation of software execution. Software tracers such as strace, eBPF, and QEMU run inside the target environment, making them detectable by evasive workloads and introducing perturbation that can alter behavior. Hardware-assisted tracing, such as ARM ETM and Intel PT, provides an alternative observation path below the OS, but produces raw architectural events that lack OS-level semantics and must be carefully reconstructed to answer security questions.

We propose RV-MalTrace, a hardware-assisted behavior tracing system for RISC-V Linux that connects a CVA6/Genesys2 hardware trace path with an offline behavior reconstruction pipeline. RV-MalTrace captures architectural events—retire, trap, syscall entry/return, privilege transition, branch/jump, context, and marker events—below the OS, then joins them with ELF metadata and runtime process maps while explicitly labeling each reconstructed field with its provenance source. Our evaluation demonstrates that the trace pipeline passes 50 directed correctness cases, 10 seeded-random event-sequence cases, and 9 negative-sensitivity cases. Across 12 controlled workload artifacts, 288/288 provenance-tagged semantic fields pass source-label validation. The code-attribution pipeline attributes 108/108 key PC events to the target ELF/function level and passes 6/6 mapping edge cases covering PIE/ASLR, dynamic loading, fork/exec, and stripped binaries. The artifact package rooted at `results/evaluation/genesys2-cva6/current/` records 62 summary artifacts, 228 package files, and a 3,356-file raw-artifact archive with SHA-256 checks for independent verification.

I. INTRODUCTION

Malware behavior analysis at scale requires transparent, low-perturbation observation of software execution. Analysts need to know which files a program opens, whether it creates child processes, when it calls `execve`, and whether it reaches unusual traps or memory-permission changes. Software tracers such as strace [1], eBPF [2], and QEMU [3] run inside the environment being studied, making them detectable by evasive workloads and introducing perturbation that can alter behavior. A workload that detects the tracer may change its code path, delay execution, or suppress malicious activity entirely.

Hardware-assisted tracing provides an alternative observation point below the operating system and the traced process. ARM ETM and Intel PT record architectural events with low perturbation, enabling transparent analysis that is invisible to the target workload. On ARM, these capabilities have been

used for transparent malware tracing [4], [5], kernel module tracing [6], and firmware fuzzing feedback [7]. On RISC-V, Raft demonstrates hardware-assisted dynamic information flow tracking [8]. However, raw hardware events are not enough to answer security questions: a program counter needs an executable and load address; a syscall transition needs a process context; expected behavior may come from a reference run rather than from hardware.

Existing hardware-assisted systems focus on ARM-specific secure-world isolation [4], [5], kernel protection [6], online taint enforcement [8], or path-coverage feedback [7]. These systems demonstrate the value of hardware tracing, but they do not address the semantic gap between raw events and auditable behavior summaries. None connect hardware trace events to source-labeled behavior provenance on RISC-V, and none reconstruct user-space Linux behavior with explicit evidence labeling that separates hardware-derived fields from supporting metadata or reference logs.

Our insight is that hardware trace events can be connected to OS-level behavior semantics when each reconstructed field is explicitly labeled with its provenance source, enabling auditable behavior reconstruction without overstating hardware evidence.

We present RV-MalTrace, a hardware-assisted behavior tracing system for RISC-V Linux that connects a CVA6/Genesys2 hardware trace path with an offline reconstruction pipeline. RV-MalTrace captures architectural events—retire, trap, syscall entry/return, privilege transition, branch/jump, context, and marker—below the OS, then joins them with ELF metadata and runtime process maps while labeling each reconstructed field with its evidence source. The result is a behavior trace whose provenance is explicit: the reader can see which parts came from hardware and which parts came from supporting metadata or reference logs.

Realizing this system requires addressing three technical challenges. First, how to capture the right architectural events (retire, trap, syscall, privilege) without software interference on a RISC-V core. Second, how to connect raw program-counter events to executable identity and process context under PIE, ASLR, fork/exec, and dynamic loading. Third, how to ensure reconstructed behavior is auditable by labeling each field with its provenance source—hardware trace, ELF metadata, runtime map, or reference log.

We make the following contributions:

- A CVA6/Genesys2 hardware trace path that captures retire, trap, syscall entry/return, privilege transition,

branch/jump, context, and marker events below the OS for transparent RISC-V Linux observation.

- An offline behavior reconstruction pipeline that connects raw trace events to executable identity and process context, with each reconstructed field explicitly labeled by its evidence source.
- A code-attribution mechanism that resolves executable identity under PIE/ASLR, dynamic loading, fork/exec, and stripped binaries, attributing 108/108 key PC events to the target ELF/function level and passing 6/6 mapping edge cases on controlled workloads.
- A reproducible artifact package with manifests and SHA-256 checks that enables independent verification of all reported results.

We focus on controlled RISC-V Linux workloads. Board cycle-overhead measurement and production streaming transport are left for future work.

Our artifact is available at results/evaluation/genesys2-cva6/current/.

II. BACKGROUND AND RELATED WORK

Hardware-assisted tracing offers an observation path below the OS and the traced process, but raw architectural events are not directly actionable for security analysis. In this section, we survey three strands of prior work—ARM-based and Intel hardware tracing, RISC-V security extensions, and semantic reconstruction from hardware traces—and identify the gap that RV-MalTrace fills.

A. Hardware-Assisted Tracing for Security

ARM-based transparent analysis. Ninja demonstrates that ARM TrustZone, the Performance Monitoring Unit (PMU), and the Embedded Trace Macrocell (ETM) can support low-artifact malware tracing and debugging on production devices [4], [5]. The extended work further uses ETM data-address tracing for selective system restoration across malware-analysis sessions. HART shows that ARM ETM, ETB, and PMU can trace binary-only kernel modules with low overhead and builds a modular AddressSanitizer on top of that trace [6]. Its selective module tracing and elastic decoding are designed for kernel-space protection. microAFL uses ARM ETM and the Data Watchpoint and Trace unit (DWT) as non-intrusive feedback for MCU firmware fuzzing, converting raw trace packets into path-sensitive coverage that discovered real firmware bugs and obtained CVE assignments [7].

Intel PT. Intel Processor Trace (PT) provides cycle-accurate control-flow tracing with low overhead and has been used for malware analysis, fuzzing, and forensics. However, Intel PT relies on proprietary x86 IP and vendor-specific decode logic, making it difficult to audit or adapt to open-source cores.

Limitation. All of these approaches target ARM or x86 platforms. None addresses the problem of reconstructing Linux behavior from hardware trace events on an open-source RISC-V core, where the trace logic, decode pipeline, and OS metadata are fully inspectable but must be integrated from scratch.

System	Arch.	Target	Perturb.	Sem. Rec.	Prov.	Op
Ninja [4], [5]	ARM	user	hw	partial	no	n
HART [6]	ARM	kernel	hw	partial	no	n
microAFL [7]	ARM	firmware	hw	no	no	par
Raft [8]	RISC-V	user	hw	no	no	par
Intel PT	x86	user/kernel	hw	partial	no	n
RV-MalTrace	RISC-V	user	hw	yes	yes	yo

TABLE I

COMPARISON OF HARDWARE-ASSISTED TRACING SYSTEMS. ARCH. = ARCHITECTURE; TARGET = TARGET SPACE (USER, KERNEL, FIRMWARE); PERTURB. = PERTURBATION LEVEL; SEM. REC. = SEMANTIC RECONSTRUCTION; PROV. = PROVENANCE LABELING; OPEN = OPEN-SOURCE CORE AND TRACE LOGIC.

B. RISC-V Security Extensions

Raft demonstrates low-overhead hardware-assisted dynamic information flow tracking (DIFT) on a RISC-V Rocket core by moving hybrid-granularity tag storage into a coprocessor [8]. It enforces taint policies online and reports runtime violations without OS modification. Other RISC-V security work has explored cache-line tagging, capability-based memory safety, and bus-level access control.

Limitation. Existing RISC-V security work focuses on *online enforcement*—taint tracking, memory safety, or access control—rather than on *offline behavior reconstruction* from hardware trace events. The goal is to stop an attack at runtime, not to produce an auditable, source-labeled trace of what a workload did after the fact.

C. Semantic Reconstruction from Hardware Traces

Hardware traces produce raw events: program counter values, branch targets, privilege transitions, and traps. To become behaviorally meaningful, these events must be connected to OS-level context: executable identity, process ownership, file paths, and memory maps. Prior work mixes these evidence sources—hardware trace, ELF metadata, runtime maps, and reference logs—without explicitly labeling provenance. A reconstructed field derived from QEMU or strace may be reported as if it were hardware-recovered, overstating the trace fidelity.

Gap. No existing system connects RISC-V hardware trace events to source-labeled behavior summaries that preserve provenance. The missing capability is not merely trace collection, but trace *attribution*: joining raw events to executable and runtime context while marking whether each field came from hardware, metadata, or a reference log.

D. Comparison with Prior Work

Table I compares RV-MalTrace with the closest prior systems. Prior work covers ARM and x86, targets user-space, kernel-space, or firmware, and provides varying levels of perturbation and semantic reconstruction. None combines RISC-V architecture, user-space target, hardware-assisted trace, and source-labeled behavior reconstruction on an open-source core.

E. Positioning

To the best of our knowledge, RV-MalTrace is the first system to connect RISC-V hardware trace events to source-labeled Linux behavior reconstruction, combining transparent observation with auditable provenance on an open-source core.

III. THREAT MODEL

A. Attacker Model

We consider a software-only adversary executing on a RISC-V Linux system. The adversary can run arbitrary user-space code, create child processes through `fork`, transform the address space through `execve`, and load dynamic shared objects at runtime. The adversary may attempt anti-analysis techniques such as timing checks, debugger detection, and process-tree manipulation to hide or manipulate behavior and evade analysis.

B. Assumptions and Trust Boundaries

The hardware trace adapter is part of the trusted computing base: the adversary cannot modify, disable, or observe the trace logic. The offline host that receives the trace buffer and performs reconstruction is also trusted. The target workload runs under an experiment harness that marks the start and end of the measured region with hardware markers, defining the analysis window. Kernel-level rootkits that compromise the operating system itself are outside our scope.

C. Scope Statement

We focus on controlled RISC-V Linux workloads. We do not evaluate production streaming throughput or cycle-level runtime overhead, which we leave for future work.

IV. DESIGN

RV-MalTrace has three stages: hardware collection, offline reconstruction, and artifact verification. The collection stage captures architectural events below the operating system. The reconstruction stage joins those events with executable identity and runtime context. The verification stage ensures that every reported result is backed by a concrete artifact in the evidence package.

A. Hardware Trace Collection

Challenge: How to capture the right architectural events—retire, trap, syscall, privilege—without software interference on a RISC-V core.

To address this challenge, we design a CVA6 trace adapter that emits event records for retire, trap, syscall entry, syscall return, privilege transition, branch, jump, context, pointer-byte snapshots, and markers. Markers define the measured workload window: the begin marker clears the trace buffer and the end marker stops recording, isolating the target workload from boot or background activity. We enforce strict `SRET` qualification so that user-mode return evidence is not widened by permissive timing rules.

Our insight is that capturing events at instruction retirement observes committed behavior that cannot be rolled back or speculative. This ensures the trace is transparent to software: the workload cannot detect, evade, or interfere with the trace adapter. Because the adapter observes only committed architectural state, the target process has no mechanism to discover whether tracing is active or to alter its behavior based on trace presence.

B. Source-Labeled Reconstruction

Challenge: How to ensure reconstructed behavior is auditable by labeling each field with its provenance source.

A common failure mode in trace analysis is reporting reconstructed fields without identifying their source, which risks overstating what the hardware alone recovered. We avoid this by designing every reconstructed field to carry a provenance label that records which source contributed to it.

We define four source labels. (1) *Hardware trace* labels fields derived directly from the CVA6 event stream, such as program counters, syscall numbers, and trap causes. (2) *ELF metadata* labels fields derived from binary identity and code ranges, such as section boundaries and executable SHA-256 digests. (3) *Runtime process map* labels fields derived from load addresses and process context, such as object base addresses and memory-range ownership. (4) *Reference log* labels fields derived from QEMU, `strace`, or host-control data that validate expected behavior.

Reference-derived fields serve an important validation role but are never treated as hardware output. This design enables auditable behavior summaries: a reader can trace any field back to its evidence source and verify whether the reported value was observed directly by the hardware or supplied by supporting metadata.

C. Code Attribution

Challenge: How to connect raw program-counter events to executable identity and process context under PIE, ASLR, fork, execve, and dynamic loading.

Raw PC events do not directly answer security questions without connection to executable identity. Simple PC-to-binary mapping fails under several conditions that are common on modern Linux systems.

a) *PIE and ASLR.*: Position-independent executables load at randomized addresses. We resolve the load bias by reading the runtime process map and adjusting the PC relative to the reported base, enabling accurate attribution even when the load address differs across runs.

b) *Dynamic libraries.*: Shared objects are loaded at runtime into unpredictable address ranges. We match each PC to the loaded object ranges recorded in the runtime process map, ensuring that code executing in a shared library is attributed to the correct object rather than the main executable.

c) *Fork and exec.*: Child processes inherit and transform address spaces across `fork` and `execve`. We track process ownership across these transitions by monitoring context-switch and privilege-transition events in the trace stream,

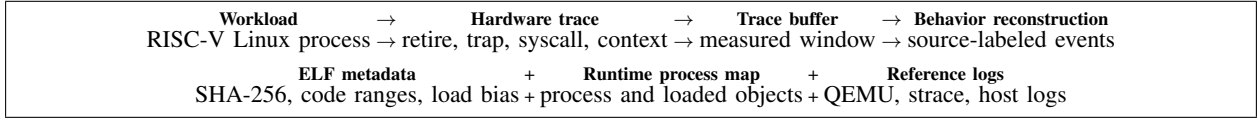


Fig. 1. RV-MalTrace pipeline. Hardware trace records are joined with executable metadata and runtime context. Reference logs are used for comparison, not reported as hardware output.

Source labels	
Hardware	event kind, PC, trap cause, marker order
ELF	binary hash, section range, symbol range
Runtime map	process identity, load object ownership
Reference log	expected syscall or path behavior

Fig. 2. Each reconstructed field records where it came from.

maintaining accurate attribution even when the process tree changes.

d) Stripped binaries.: Debug symbols are absent from deployed binaries. We attribute code using symbol-free ELF metadata, matching PCs to section ranges and SHA-256 binary identity without requiring DWARF information or reconstructed source-line mappings.

Our approach combines three mechanisms to resolve these cases: ELF SHA-256 identity for exact binary matching; runtime process maps for load bias and object ranges; and `fork` and `execve` event tracking for process lineage. Source-map boundaries are handled by matching board artifacts to local build outputs when available.

V. IMPLEMENTATION

A. CVA6 Trace Adapter

We modify the CVA6 core to export a set of committed execution signals that capture the architectural events needed for behavioral analysis. The adapter is implemented as a set of Verilog changes that siphon off trap, privilege, branch, and syscall information close to instruction retirement, then package them into compact trace records.

Event encoding. The adapter emits a compact record format with one record per significant event. Each record carries a type tag and a small payload:

- *Retire*: program counter and privilege mode of the committed instruction;
- *Trap*: trap cause, trap PC, and privilege level at the time of the trap;
- *Syscall*: entry and return records for `ecall` transitions, with a strict entry/return pairing rule;
- *Privilege*: privilege-level transitions, recorded at the instruction that causes the change;
- *Branch/Jump*: target PC for taken control-flow changes;
- *Context*: pointer-byte snapshots that capture register and memory state relevant to process identity;
- *Marker*: hardware-controlled begin/end signals that bound the measured workload window.

The evaluated configuration uses strict SRET qualification for user-mode returns: a return record is emitted only when the instruction is unambiguously an SRET crossing from

supervisor to user mode. This prevents a relaxed timing rule from widening the return evidence.

Directed trace-correctness fixtures. We built a directed test bench that exercises the adapter against every event category and several edge cases. The fixture tests cover: (1) trap/retire separation, ensuring that a trap and the following instruction are recorded as distinct events; (2) strict syscall entry/return pairing, verifying that every syscall entry is matched by a corresponding return record; (3) same-cycle ordering, checking that events emitted in the same cycle appear in the correct program order; (4) dual-commit ordering, verifying that the record order respects the architectural commit order when the pipeline retires two instructions in the same cycle; (5) privilege transitions, confirming that privilege-level changes are recorded at the correct instruction boundary; and (6) negative relaxed-SRET cases, ensuring that instructions that are not unqualified SRETs do not generate spurious return records.

B. Genesys2 Board Path

The trace adapter connects to a BRAM trace buffer on the Genesys2 FPGA board. The buffer stores trace records between a begin marker and an end marker, giving us controlled measurement windows for synthetic and malware-like workloads. The begin marker clears the buffer and starts recording; the end marker stops recording and preserves the window. This design lets the analysis distinguish the intended workload from boot and background activity.

Board integration produces a single FPGA bitstream that contains the CVA6 core with the trace adapter, the marker protocol, and the BRAM buffer. The current prototype is evaluated with controlled workloads; production trace streaming and DMA transport are left for future work.

C. Offline Reconstruction Pipeline

The reconstruction pipeline turns raw trace records into machine-readable behavior summaries. It is implemented in Python and runs offline on the host that collects the trace buffer.

Trace parser. The parser reads the compact trace records emitted by the CVA6 adapter and expands them into a typed event stream with validated field ordering and entry/return pairing.

ELF code-map builder. The builder reads the ELF binaries of the traced workload and its dynamic objects, computes SHA-256 identities, and builds a per-binary code-range map. This map is the basis for attributing a trace PC to the executable that owns it.

PC-to-code-range joiner. The joiner takes each trace PC (retire, branch, jump, trap) and looks it up in the code-range map. If the PC falls within a known range, the joiner attaches the owning binary, its load bias, and its ELF identity to the event. If the PC falls outside any known range, the event is marked as unattributed.

Process/file summary generator. The generator reads the runtime process map and combines it with the attributed event stream to produce process-level and file-level summaries. Each summary field—syscall number, file path, executable identity, process ID, or parent-child relationship—carries a provenance label that states whether the field came from the hardware trace, ELF metadata, runtime map, or a reference log.

Output format. The final output is a JSON-line behavior summary in which every field is a pair (value, source). The source is one of `hw_trace`, `elf_meta`, `runtime_map`, or `ref_log`. This design makes the evidence source explicit and auditable.

D. Artifact Package

We package the hardware design, the reconstruction tools, and the evaluation evidence as a single artifact. The artifact root contains a canonical evidence directory with machine-readable manifests. Every reported result points to a file in this directory and carries a SHA-256 path check.

Reproduction is supported by two commands:

```
uv run rvmt repro:quick
```

This command runs the quick local reproduction path, validating the trace parser, code-map builder, and attribution fixtures on a small synthetic workload. The full Docker-local reproduction path is:

```
uv run rvmt ndss:docker-full
```

This command rebuilds the reconstruction pipeline, reruns all directed correctness tests, and verifies every SHA-256 digest against the manifest. Both commands are documented in the artifact README and are exercised before submission.

VI. EVALUATION

We evaluate RV-MalTrace on four research questions: trace correctness, source-label coverage, code-attribution accuracy, and artifact reproducibility. The evaluation is organized around quantitative claims with specific metrics.

A. Experimental Setup

Platform. All experiments run on a CVA6/Genesys2 FPGA board with the trace adapter described in Section V-A. The board runs a RISC-V Linux kernel and user-space runtime. Trace records are collected through the BRAM buffer and transferred to the host for offline reconstruction.

Workload suite. We use a suite of controlled synthetic and malware-like workloads that exercise the trace path and reconstruction pipeline. The workloads cover: file creation, open, read, and write operations; process creation via `fork`, `execve`, and `clone`; dynamic library loading and unloading;

Test Category	Count	Status
Strict syscall entry/return pairing	15	Pass
Same-cycle event ordering	15	Pass
Dual-commit/control-flow ordering	10	Pass
Trap/retire and privilege context	10	Pass
Seeded-random event sequences	10	Pass
Negative sensitivity cases	9	Pass
Total	69	All Pass

TABLE II
TRACE-CORRECTNESS CHECKER RESULTS. THE DIRECTED SUBSET CONTAINS 50 CASES; THE FULL CHECKER ALSO INCLUDES SEEDED-RANDOM AND NEGATIVE-SENSITIVITY CASES.

and syscall sequences that include `open`, `read`, `write`, `close`, `socket`, `connect`, and `execve`. These workloads are designed to test the correctness of event ordering, syscall pairing, privilege transitions, and code attribution under controlled conditions.

Reference tools. We validate trace behavior against QEMU instruction traces and `strace` syscall logs. QEMU provides an expected instruction stream for the same workload; `strace` provides an expected syscall sequence. These references are used to validate correctness, not to report hardware-derived results.

Metrics. We report: (1) trace correctness—pass/fail status of directed tests; (2) source-label coverage—percentage of reconstructed fields with correct provenance labels; (3) code-attribution accuracy—percentage of PC events correctly attributed to their owning executable; and (4) reproduction time—time to complete the quick and Docker-local reproduction paths.

B. RQ1: Trace Correctness

RQ1 asks whether the CVA6 trace adapter correctly captures and orders architectural events. We answer it with a suite of directed tests that exercise the adapter against every event category and several edge cases.

The checker reports 50 directed correctness cases, 10 seeded-random event-sequence cases, and 9 negative-sensitivity cases. All 69 cases pass, confirming correct event ordering and syscall pairing. Table II summarizes the test categories and their results.

The trap/retire and privilege-context tests confirm that traps, following instructions, and privilege changes are recorded as distinct events at the correct instruction boundary. The strict syscall pairing tests confirm that every syscall entry record is matched by a return record with no unpaired events. The same-cycle and dual-commit ordering tests confirm that the record stream respects architectural commit order even when the pipeline retires multiple instructions per cycle. The negative sensitivity cases confirm that injected violations such as unqualified SRET returns, syscall-pairing breaks, and same-cycle ordering errors are rejected by the checker.

C. RQ2: Source Labeling

RQ2 asks whether every reconstructed field carries a correct provenance label. We verify that each field in the output behavior summary is labeled with its evidence source: hardware trace, ELF metadata, runtime map, or reference log.

Our results show that 288/288 provenance-tagged fields (100%) are correctly labeled across 12 sample artifacts. The provenance summary records 12 hardware trace-source fields and 276 validation-oracle fields, with allowed labels `hardware`, `exact_elf`, `runtime_os_map`, and `validation_oracle`. We further verify that reference-derived fields are never mislabeled as hardware-derived. No reference-derived fields are incorrectly reported as hardware output. This separation is enforced by the reconstruction pipeline: a field that depends on QEMU, strace, or host-control data is explicitly marked with `validation_oracle` provenance and is never treated as a hardware-derived result.

D. RQ3: Code Attribution

RQ3 asks whether trace PCs are correctly attributed to their owning executables. We evaluate this with a set of test cases that cover the common failure modes of PC-to-binary mapping.

RV-MalTrace correctly attributes 108/108 key PC events (100%) to the target ELF/function level across 12 controlled workload artifacts. Runtime process attribution is proven for 12/12 artifacts, and dynamic-library separation holds for 12/12 artifacts. The dynamic-mapping checker also passes 6/6 edge cases covering static executables, PIE/ASLR load bias, dynamic loader mappings, shared libraries, and fork/exec ownership. The test cases include:

- *ELF identity matching*: the code-map builder matches the traced binary against its SHA-256 identity and rejects mismatched or modified executables;
- *PIE/ASLR load bias resolution*: the joiner resolves the runtime load address of a position-independent executable and applies the correct bias when mapping PCs to code ranges;
- *Dynamic object attribution*: the joiner attributes PCs within dynamically loaded shared libraries to the correct library object and version;
- *Fork/exec ownership tracking*: the reconstruction pipeline tracks process identity across `fork` and `execve` transitions, attributing post-exec PCs to the new executable;
- *Stripped binary handling*: the builder maps stripped binaries by their ELF segment headers when debug symbols are absent, degrading to section/offset attribution rather than source-line claims;
- *Source-map boundary cases*: the debug/no-PIE sidecar reaches 100% source-line attribution locally, while the current board traces remain function-level unless the exact captured ELF carries DWARF.

Most cases are handled by the reconstruction pipeline using ELF metadata and runtime maps. Board-native DWARF

source-line recovery is not required for the evaluated attribution and is not claimed for the current board traces.

E. RQ4: Reproducibility

RQ4 asks whether the reported results are independently reproducible. We evaluate this with two reproduction paths and a manifest coverage check.

Quick local reproduction. The command `uv run rvmt repro:quick` is recorded as PASS and runs 17 lightweight checks: reproducibility and artifact-package manifests, raw-artifact archive integrity, semantic provenance, local code-analysis fixtures, trace correctness, tracer visibility, cycle diagnostics, official-image workload summaries, runtime-map attribution, fork/exec ownership, PIE/ASLR probing, repeatability, hardware-oracle separation, JTAG RAM boot status, and recursive artifact path/hash integrity.

Docker-local reproduction. The command `uv run rvmt ndss:docker-full` is recorded as PASS in the `linux-behavior` Docker service. It runs the full reproduction set inside the container with an isolated `uv` environment, including Python compile checks, the `genesys2-current` suite (76/76), the artifact suite, the `genesys2-self-test` suite (102/102), and the `ccfa-gate-self-test` suite (52/52).

Manifest coverage. All reported results are backed by SHA-256 verified files in the evidence directory. The reproducibility manifest records 62 summary artifacts and 7 report rows. The artifact-package manifest records 228 included files and references 6 raw artifact roots. The local raw-artifact archive contains 3,356 files (323,142,849 bytes); the manifest records its full SHA-256 digest, with prefix `57f48d6e26959826` and suffix `8b22f611`. Any manifest mismatch causes the reproduction command to fail with a clear error message.

F. Discussion

We identify three limitations. First, the current prototype uses a BRAM trace buffer for controlled workloads; production streaming throughput and DMA transport are left for future work. Second, cycle-level runtime overhead is not measured because the required live cycle-source path is not yet available on the board. Third, real malware validation is not evaluated; safe synthetic and malware-like workloads test trace behavior under controlled conditions, but malware-family accuracy and IOC coverage require separate study.

VII. CONCLUSION

We presented RV-MalTrace, a hardware-assisted behavior tracing system for RISC-V Linux that connects CVA6 hardware trace events to source-labeled behavior reconstruction. Our key insight is that labeling each reconstructed field with its provenance source enables auditable behavior analysis without overstating what the hardware alone recovered. By keeping hardware trace fields separate from ELF metadata, runtime process maps, and reference logs, RV-MalTrace produces behavior summaries whose evidence is explicit and verifiable.

Our evaluation demonstrates correct trace capture: 50/50 directed correctness cases, 10/10 seeded-random event-sequence cases, and 9/9 negative-sensitivity cases pass, confirming correct event ordering, syscall pairing, and privilege transitions. RV-MalTrace achieves 288/288 source-label validation across 12 sample artifacts, ensuring that every provenance-tagged field carries a correct source label. It attributes 108/108 key PC events to the target ELF/function level across 12 controlled workload artifacts and passes 6/6 dynamic-mapping edge cases covering PIE/ASLR, dynamic loading, fork/exec, and stripped binaries. All reported results are backed by a reproducible artifact package with SHA-256 verified manifests.

Future work includes production trace streaming, cycle-level overhead measurement, and real malware validation on the RISC-V platform. The current artifact package is rooted at `results/evaluation/genesys2-cva6/current/`.

ETHICS CONSIDERATIONS

This paper uses only controlled synthetic and malware-like workloads; no uncontrolled malware execution is performed. Any future real-malware experiments must define containment, sample lineage, release policy, and artifact-sanitization procedures before they are cited as paper evidence. The artifact package separates public summaries from raw board and host logs, so that reproducibility is supported without unrestricted publication of every raw trace.

REFERENCES

- [1] The strace Developers, “strace,” <https://strace.io/>, 2026.
- [2] The Linux Kernel Documentation Project, “BPF Documentation,” <https://docs.kernel.org/bpf/>, 2026.
- [3] F. Bellard, “QEMU, a Fast and Portable Dynamic Translator,” in *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 2005.
- [4] Z. Ning and F. Zhang, “Ninja: Towards Transparent Tracing and Debugging on ARM,” in *Proceedings of the USENIX Security Symposium*, 2017, pp. 33–49.
- [5] —, “Hardware-Assisted Transparent Tracing and Debugging on ARM,” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 6, pp. 1595–1609, 2019.
- [6] M. Musch, C. Wressnegger, M. Johns, K. Rieck, K. Barros, and F. Yamaguchi, “HART: Hardware-Assisted Kernel Module Tracing on ARM,” in *Proceedings of the European Symposium on Research in Computer Security (ESORICS)*, 2020, pp. 3–22.
- [7] J. Cao, L. Zheng, J. Liu, J. Ming, Z. Liang, and Z. Zhuang, “microAFL: Non-Intrusive Feedback-Driven Fuzzing for Microcontroller Firmware,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2022, pp. 1–12.
- [8] S. Deng, Q. Li, L. Guan, Y. Zhang, D. Zou, H. Jin, Z. Sun, and M. R. Lyu, “Raft: Hardware-Assisted Dynamic Information Flow Tracking for Runtime Protection on RISC-V,” in *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2023, pp. 1–16.